UNIVERSAL CODE SCANNER

12 · Pruebas y calidad

Documentación de sistema

Versión analizada: **1.1.0+2**
Commit: **c1f9878**
Fecha del documento: **2026-08-27**

Documento generado desde el Markdown de **docs/system-documentation/** con **tool/build_system_documentation_pdf.py**. El Markdown es la fuente: no editar este PDF.

Contenido

1. Conclusión primero
2. 1 · Qué se ejecutó, con su salida real
3. 2 · Inventario de la suite
4. 3 · Las pruebas que más valen
5. 4 · Patrones de prueba del repositorio
6. 5 · El catálogo de regresión que nadie ejecuta
7. 6 · Qué queda sin cubrir
8. 7 · Cobertura
9. 8 · Análisis estático y gates
10. 9 · Verificación de esta entrega
11. 10 · Propuesta priorizada
12. 11 · Lo que está bien y conviene proteger

Conclusión primero

La suite es **pequeña pero bien elegida**: 57 casos que cubren con precisión los puntos donde un error sería costoso —criptografía, migración, importación, cancelación y estado del escáner— y dejan sin cubrir las pantallas grandes y el camino real de la cámara, que no se pueden probar sin hardware.

Hay un hallazgo que merece nombrarse antes que nada: **el catálogo de regresión de `test_assets/` no lo ejecuta ninguna prueba.** Ocho imágenes con su carga esperada, un manifiesto, y un validador que comprueba únicamente que los archivos existan. Ninguna prueba Dart los decodifica ni los compara. Es el hallazgo R-01.

---

1 · Qué se ejecutó, con su salida real

Sobre el commit `c1f9878`, con Flutter 3.44.7 y Dart 3.12.2 en Windows 11:

`flutter analyze --fatal-infos`

```
Analyzing universal-code-scanner...
No issues found! (ran in 100.8s)
```

Código de salida: `0`.

`flutter test`

```
00:24 +57: All tests passed!
```

Código de salida: `0`. 57 de 57 en verde, en 24 segundos.

`python tool/validate_structure.py`

Su resultado se registra al final de este documento, en el apartado 9, junto con la verificación completa de la entrega.

Estas dos primeras ejecuciones son la **línea base previa a cualquier cambio de este trabajo**. Coinciden con lo que declara `../../VALIDATION.md` y con la insignia del `README.md`.

---

2 · Inventario de la suite

20 archivos de prueba, 58 casos declarados. `flutter test` ejecuta los 57 de `test/`; el de `integration_test/` necesita un dispositivo.

Archivo	Casos	Qué protege
`test/core/payload_cipher_test.dart`	5	Sobre versionado, sobre heredado, llave ausente, manipulación del texto cifrado, versión futura
`test/core/data_maintenance_test.dart`	3	Rotación correcta, aborto ante carga incompleta, limpieza de llave huérfana
`test/scan_security_analyzer_test.dart`	9	HTTPS normal, HTTP, correo en la consulta, credenciales, Punycode, esquema no permitido, `tel:`, Wi-Fi, OTP

`test/content_interpreter_test.dart`	6	Wi-Fi con escapes, vCard, GS1, EMVCo, `www.`, ISBN
`test/scan_record_test.dart`	3	Serialización completa, contenido binario, detección de sensibles
`test/services/import_service_test.dart`	4	Vista previa, esquema futuro, duplicados internos, JSON profundo
`test/features/scan_status_bar_test.dart`	5	Barra en movimiento, movimiento reducido, pausa, cámara no disponible, barrido del marco
`test/core/handheld_frame_test.dart`	3	Móvil, umbral exacto, pantalla ancha centrada
`test/core/localization_test.dart`	4	Solo español, etiquetas resueltas, rechazo de inglés, claves inglesas conservadas
`test/services/scan_feedback_test.dart`	4	Tono y vibración, canales independientes, degradación al sonido del sistema, liberación
`test/core/async_write_queue_test.dart`	2	Orden de envío, un fallo no bloquea la cola
`test/core/cancellation_token_test.dart`	2	Cancelación determinista, fracción acotada
`test/services/history_repository_test.dart`	1	`replaceAll` atómico
`test/services/content_parser_registry_test.dart`	1	Parser propio aislado y removible
`test/core/recovery_service_test.dart`	1	El paquete no lleva llaves ni texto claro
`test/core/schema_migrator_test.dart`	1	Migración idempotente y no destructiva
`test/core/diagnostics_test.dart`	1	La exportación excluye mensajes y valores sensibles
`test/core/accessibility_test.dart`	1	Centro de recuperación con texto al 200 %
`test/inventory_session_test.dart`	1	Unidades y serialización
`integration_test/app_launch_test.dart`	1	Arranque normal o seguro. **No la ejecuta `flutter test`**

Distribución por tipo:

Tipo	Casos	Porcentaje
Unitarias puras	39	68 %
De widget (`testWidgets`)	18	32 %

De integración	1	Fuera de `flutter test`

---

3 · Las pruebas que más valen

No todas las pruebas valen lo mismo. Estas cinco son las que impedirían un desastre.

1 · «a missing decryption key is never replaced silently» (`payload_cipher_test.dart`). Comprueba que tras olvidar la llave, tanto `decryptJson` como `encryptJson` lanzan, y que **el proveedor sigue sin la llave** después de ambos intentos. Sin esta propiedad, un descifrado fallido seguido de una escritura destruiría el historial para siempre.

2 · «failed rotation removes a newly generated orphan key» (`data_maintenance_test.dart`). Usa un `_TrackingKeyProvider` que envuelve al de memoria para observar qué llave se creó y cuál se olvidó. Verifica que la llave nueva se creó, se olvidó, ya no se puede leer, y que el metadato de llave activa **no cambió**.

3 · «tampered authenticated ciphertext is rejected» (`payload_cipher_test.dart`). Invierte un bit del texto cifrado y comprueba que el descifrado falla. Es la prueba de que GCM está haciendo su trabajo.

4 · «diagnostic export excludes error messages and sensitive values» (`diagnostics_test.dart`). Registra un `StateError` cuyo mensaje contiene literalmente una contraseña, un código OTP y un dominio, y verifica que ninguno aparece en la exportación —pero que `StateError` sí. Prueba una propiedad de privacidad, no un comportamiento.

5 · «excessively deep JSON is rejected before persistence» (`import_service_test.dart`). Construye un JSON con 34 niveles de anidamiento y comprueba que se rechaza. Protege contra un archivo hostil que agotara la pila.

Una sexta merece mención por lo que revela del cuidado del equipo: `test/core/accessibility_test.dart` levanta el Centro de recuperación con `TextScaler.linear(2)` y comprueba las guías `labeledTapTargetGuideline` y `textContrastGuideline` de Flutter. Y lo hace con `tester.runAsync` porque —según su propio comentario— la base de datos usa temporizadores reales que no funcionan bajo el reloj falso de `testWidgets`. Ese detalle costó un defecto real, registrado en `VALIDATION.md`: la prueba se colgaba diez minutos.

---

4 · Patrones de prueba del repositorio

Merece documentarlos porque quien añada una prueba tendrá que seguirlos.

Base de datos en memoria

```
final AppDatabase database = await AppDatabase.openTemporary();
addTearDown(database.close);
```

Nunca se toca el disco. `openTemporary` genera un nombre único con `microsecondsSinceEpoch`, así que dos pruebas del mismo archivo no comparten estado.

Llaves en memoria

```
final PayloadCipher cipher = PayloadCipher(keyProvider: MemoryEncryptionKeyProvider());
```

`MemoryEncryptionKeyProvider` existe para esto y para el modo temporal. Sin él, las pruebas de cifrado necesitarían un Keychain.

Preferencias en memoria

```
setUp(() {
 TestWidgetsFlutterBinding.ensureInitialized();
 SharedPreferencesAsyncPlatform.instance = InMemorySharedPreferencesAsync.empty();
});
```

Requiere la dependencia de desarrollo `shared_preferences_platform_interface 2.4.2`, que está en `pubspec.yaml` exactamente por esto. Cuatro casos fallaban por no registrarla; el defecto está en `VALIDATION.md`.

Nunca `pumpAndSettle` sobre una animación infinita

```
// Never pumpAndSettle: while it is scanning the bar animates forever, which
// is exactly the state under test.
await tester.pump();
```

El comentario está en el propio `scan_status_bar_test.dart`. `pumpAndSettle` espera a que no queden animaciones; la barra indeterminada nunca para.

Comprobar la animación por el contador de callbacks

```
await pumpOverlay(active: true, animate: true);
expect(tester.binding.transientCallbackCount, greaterThan(0));

await pumpOverlay(active: false, animate: true);
await tester.pump(const Duration(milliseconds: 16));
expect(tester.binding.transientCallbackCount, 0);
```

Es la forma correcta de verificar que una animación **está parada**: contar los callbacks transitorios en vez de inspeccionar píxeles.

Dobles por interfaz, no por biblioteca de mocks

No hay `mockito` ni `mocktail` en `pubspec.yaml`. Los dobles se escriben a mano implementando la interfaz: `_FakeTonePlayer`, `_TrackingKeyProvider`, `_TestParser`. Es coherente con las fronteras estrechas descritas en 09-apis-and-integrations.md: interfaces de dos a cuatro miembros no necesitan generación de código.

---

5 · El catálogo de regresión que nadie ejecuta

Este es el hallazgo principal de este documento.

`test_assets/` contiene ocho imágenes PNG y un `manifest.json` que declara, para cada una, su **carga esperada**, su formato y su escenario:

Archivo	Escenario	Carga esperada
`qr/url_https.png`	`qr`	Una URL de ejemplo
`qr/wifi.png`	`qr`	Un QR de Wi-Fi con contraseña
`qr/vcard.png`	`qr`	Una vCard completa
`gs1/gs1_text.png`	`gs1`	Identificadores GS1 con GTIN, vencimien to y lote
`damaged/url_partially_damaged.png`	`damaged`	La misma URL, con daño parcial
`low_light/wifi_low_light.png`	`low_light`	El mismo Wi-Fi, con poca luz

`multiple_codes/url_and_vcard.png`	`multiple_codes`	**Dos** cargas en una imagen
`curved/url_warped.png`	`curved_simulation`	La misma URL, deformada

`tool/validate_structure.py::check_test_assets` comprueba tres cosas: que el manifiesto exista, que tenga `schemaVersion: 1` y una lista `items`, y que **cada archivo declarado exista en disco**.

Lo que **no** comprueba: nada del contenido. No decodifica ninguna imagen, no compara con `expectedPayload`, no verifica el formato.

Y la verificación decisiva:

```
grep -rn "test_assets" test/ integration_test/ lib/ | wc -l
# → 0
```

Ninguna prueba Dart lee ese directorio. Los ocho escenarios —daño parcial, poca luz, deformación, varios códigos en una imagen— están preparados, versionados y documentados, y **nunca se ejecutan**. Su única función actual es alimentar la matriz manual de `../quality/DEVICE_TEST_MATRIX.md`, que los menciona como «recursos reproducibles».

Además, seis carpetas contienen únicamente un `README.md` sin imágenes: `aztec/`, `datamatrix/`, `ean/`, `micro_qr/`, `pdf417/` y `upc/`. Esto **sí está declarado** y es una decisión defendible: `IMPLEMENTATION_STATUS.md` dice que las demás simbologías «requieren capturas reales verificadas antes de incorporarse», y `test_assets/README.md` añade que «no se inventan muestras que puedan entregar falsos positivos».

Inferencia basada en el código: una prueba que use `MobileScannerController.analyzeImage` sobre estas imágenes necesitaría el motor de plataforma, que no existe en `flutter test`. Sería una prueba de integración con dispositivo, no unitaria. Esa es probablemente la razón de que no exista; **no está documentada en el repositorio**.

---

6 · Qué queda sin cubrir

Sin cobertura de prueba automatizada

Área	Líneas aproximadas	Por qué no está cubierta	¿Es aceptable?
`ScannerScreen`	638	Necesita cámara de plataform a	Sí, salvo la lógica pura
`InventoryScreen`	491	Idem	Sí, salvo la lógica pura
`SettingsScreen`	378	Widget grande, sin lógica críti ca propia	Parcialmente
`ScanResultsSheet` y `ScanRe cordCard`	318	**Contiene la decisión de en mascarado y la de confirma ción**	**No**
`GeneratorScreen`	310	`_escape` y `_normalizeUrl` so n puras y no se prueban	**No**
`HistoryScreen`	284	Filtrado en memoria, probable con widget test	Parcialmente
`ExportService`	164	`_encodeCsv` y `_encodeXlsx` s on funciones puras	**No**

`PdfPageRenderer`	115	Necesita `pdfrx` y sistema de archivos	Sí
`RecoveryScreen`	139	Cubierto indirectamente por la prueba de accesibilidad	Parcialmente
`FormatsScreen`	170	Puramente informativa	Sí
`InventoryStore` y `ScanStore`	248	Solo `AsyncWriteQueue` se prueba por separado	**No**
`EncryptionMetadataRepository`	30	La migración desde preferencias no tiene prueba propia	**No**

Casos límite conocidos sin prueba

Cada uno corresponde a una decisión real del código:

1. **La migración del historial desde preferencias.** Es el flujo con más precauciones del repositorio —respaldo cifrado, verificación dentro de la transacción, borrado solo al final— y **no tiene ninguna prueba**. R-34.
2. **La actualización perezosa de sobres al leer.** `load` reescribe los sobres antiguos; ninguna prueba lo comprueba.
3. **`_trim()` al superar 5 000 registros.** No hay prueba de que borre los más antiguos.
4. **`pruneOlderThan`.** No hay prueba del filtro por fecha.
5. **El filtro de contenido sensible en `_persistAndShow`.** Se prueba que `isSensitive` sea `true` para OTP y Wi-Fi, pero **no que el registro quede fuera del historial**.
6. **La deduplicación de dos segundos del escáner.**
7. **`_escape` y `_normalizeUrl` del generador**, ambas puras y triviales de probar.
8. **`_encodeCsv`**: el BOM, el `\r\n` y el entrecomillado universal son decisiones concretas y comprobables sin plataforma.
9. **`resetNonSensitive`**: que deje intactas exactamente tres claves.
10. **`ContentInterpreter`**: 6 casos para 20 ramas. Sin cobertura para MeCard, vEvent, `mailto:`, `tel:`, `sms:`, `geo:`, `otpauth:`, cripto, Swiss QR, EPC/SEPA, AAMVA, binario ni producto.

---

7 · Cobertura

`flutter test --coverage` genera `coverage/lcov.info`, y la CI lo sube como artefacto. **No hay umbral mínimo declarado en ningún sitio**: ni en el workflow, ni en `validate_structure.py`, ni en la documentación. Nada falla por bajar la cobertura. R-35.

No se ejecutó `--coverage` en este análisis, así que **no se dispone de un porcentaje medido**. Lo que sí se puede afirmar contando líneas:

Capa	Líneas	Con prueba directa
`core/security`	435	Sí, extensiva
`core/database`	91	Solo `SchemaMigrator`
`core/performance`	58	Sí, ambas clases

`core/recovery`	197	Solo `exportBundle`
`models`	398	Parcial
`services`	1 401	Parcial: importación, feedback, historial
`state`	270	No
`features`	3 072	Solo `ScanStatusBar` y `ScannerOverlay`

Inferencia: la cobertura de línea estará dominada por `features/`, que es el 45 % del código y casi no tiene pruebas. Un porcentaje global sería engañoso: lo relevante es que **el 100 % de la criptografía tiene prueba y el 0 % de los stores también**.

---

8 · Análisis estático y gates

`analysis_options.yaml`

```
include: package:flutter_lints/flutter.yaml
linter:
 rules:
   always_declare_return_types: true
   avoid_print: true
   prefer_final_locals: true
   use_build_context_synchronously: true
```

Cuatro reglas sobre `flutter_lints 6.0.0`. Dos merecen comentario:

- **`avoid_print`** es un control de privacidad, no solo de estilo. Se combina con `check_sensitive_logging` del validador.
- **`use_build_context_synchronously`** es el que obliga al patrón `if (!mounted) return;` y `if (context.mounted)` que aparece en todas las pantallas. Veintidós avisos de esta clase se corrigieron durante la validación de 1.0.0, según `VALIDATION.md`.

La CI usa `--fatal-infos`, que convierte incluso los avisos informativos en fallo. Es el nivel más estricto disponible.

Discrepancia real: `tool/run_quality_gate.sh` ejecuta `flutter analyze` **sin** `--fatal-infos`. Un desarrollador que use la puerta de calidad local puede pasar en verde y fallar en la CI. R-06.

Los diez gates de `validate_structure.py`

Ya enumerados en 04-code-map.md. Lo que revelan sobre las prioridades del proyecto:

Gate	Qué considera irrompible
`check_interface_version`	Que la versión mostrada al usuario sea la real. Nació de un defe cto concreto
`check_sensitive_logging`	Que nunca se registre una carga escaneada
`check_absolute_paths`	Que no se filtre una ruta del entorno de origen
`check_markdown_links`	Que ningún enlace de la documentación esté roto
`check_imports`	Que ningún `import` interno apunte a un archivo inexistente

`check_test_assets`	Que los recursos declarados existan
`check_source_sbom`	Que el SBOM de fuente exista y sea CycloneDX

`check_markdown_links` es especialmente útil para este trabajo: **valida los enlaces relativos de los 20 documentos nuevos gratis**, sin escribir un verificador aparte.

Los dos workflows

`flutter-ci.yml` corre en `push` a `main` y `develop`, en cada pull request y a petición. Dos trabajos:

Trabajo	Runner	Pasos
`quality`	`ubuntu-latest`	Validación offline con `--require-lock`, `bootstrap.py`, `pub deps`, `analyze --fatal-infos`, `test --coverage`, SBOM, licencias, `build web --release`, `build apk --debug`, checksums, subida de artefactos
`apple`	`macos-latest`	`bootstrap.py --platforms ios,macos`, `build ios --release --no-codesign`, `build macos --release`

El trabajo `apple` **no ejecuta la prueba de integración**, con el motivo escrito como comentario al final del archivo: los runners macOS de GitHub no tienen sesión gráfica interactiva, así que la aplicación arranca sin llegar a establecer la conexión de depuración. Es un ejemplo de degradación declarada en vez de silenciosa.

---

9 · Verificación de esta entrega

Ejecutado tras escribir la documentación y añadir los comentarios de código, para comprobar que nada de este trabajo rompió la base verde. La salida real está en el informe final; el resumen es:

Comprobación	Antes de los cambios	Después
`flutter analyze --fatal-infos`	`No issues found!`	`No issues found!`
`flutter test`	`+57: All tests passed!`	`+57: All tests passed!`
`python tool/validate_structure.py`	Sin errores	Sin errores
Enlaces relativos de la documentación	—	Validados por `check_markdown_links`
Mojibake	—	`grep -rl $'\xc3\x83'` sin resultados

---

10 · Propuesta priorizada

Informativo. **No se implementó ninguna** en este trabajo.

#	Propuesta	Valor	Esfuerzo

1	Ejecutar el catálogo de `test_assets/` en una prueba de integración con dispositivo, comparando contra `expectedPayload`	**Alto**: hoy ocho escenarios están preparados y sin usar	Media jornada
2	Probar la migración del historial desde preferencias, incluidos el respaldo cifrado y el fallo	**Alto**: es el flujo con más riesgo de pérdida de datos	3 horas
3	Probar que un registro sensible **no** entra en el historial	**Alto**: es una garantía de privacidad enunciada públicamente	1 hora
4	Probar `_trim` y `pruneOlderThan`	Medio	2 horas
5	Alinear `run_quality_gate.sh` con la CI añadiendo `--fatal-infos`	Medio, coste cero	1 minuto
6	Ampliar `ContentInterpreter` a las 20 ramas	Medio	3 horas
7	Probar `_encodeCsv`, `_escape` y `_normalizeUrl`	Medio, son funciones puras	1 hora
8	Probar `ScanRecordCard`: enmascarado y confirmación	Medio	2 horas
9	Declarar un umbral de cobertura y hacerlo fallar la CI	Bajo hoy, alto a largo plazo	1 hora
10	Añadir un paso de CI que compare el SBOM contra una base de CVE	Medio	Media jornada

---

11 · Lo que está bien y conviene proteger

- **La suite entera corre en 24 segundos** y no toca disco, red ni plataforma. Es una suite que la gente ejecutará.
- **Las pruebas de criptografía prueban propiedades, no implementaciones.** «Una llave ausente nunca se reemplaza en silencio» seguirá siendo válida aunque se reescriba `PayloadCipher`.
- **Los comentarios dentro de las pruebas explican el porqué**, no el qué. El de `pumpAndSettle` y el de `tester.runAsync` ahorran horas a quien los lea.
- **La CI compila cuatro plataformas**, no solo la de desarrollo.
- **El validador estructural es un gate propio y barato** que corre offline y en menos de un segundo.
- **La degradación está declarada, no oculta**: `generate_sbom.py` marca su cobertura real y el workflow explica por qué omite la prueba de integración.

Continúa en 13 · Despliegue y operación.